

Canonical Labelling and Graph Isomorphism: A C++ Reimplementation of Nauty

Auteur: Rodolphe Prévot

Promoteur: Robin Petit

Directeur: Gwenaël Joret

Université libre de Bruxelles
Département d'Informatique

Septembre 2026



1 Contexte

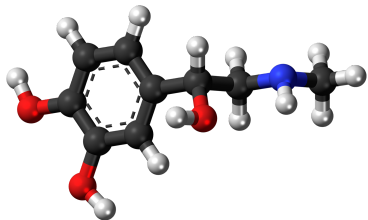
2 Nauty

3 Conclusion

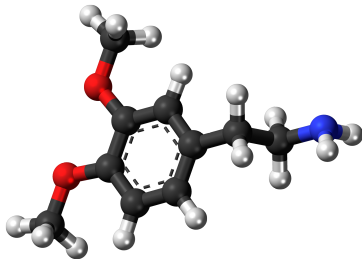
Contexte



Isomorphisme



$\equiv$?



Structurellement les mêmes?

Exemple (1/2)

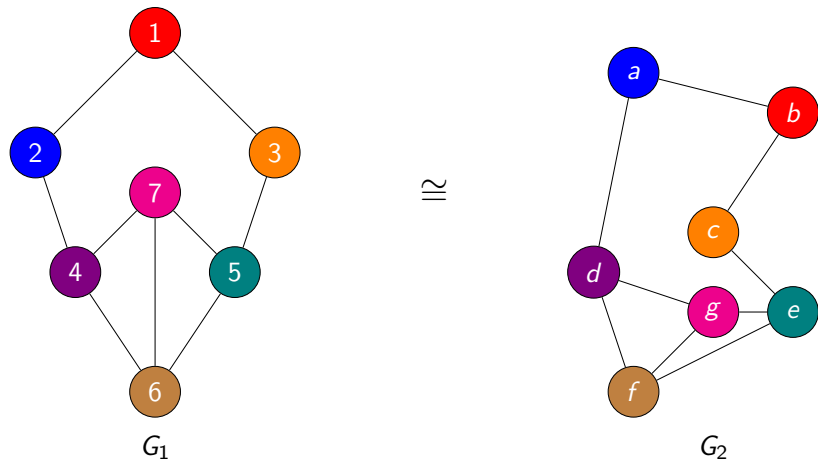


Figure 1: Isomorphisme

Exemple (2/2)

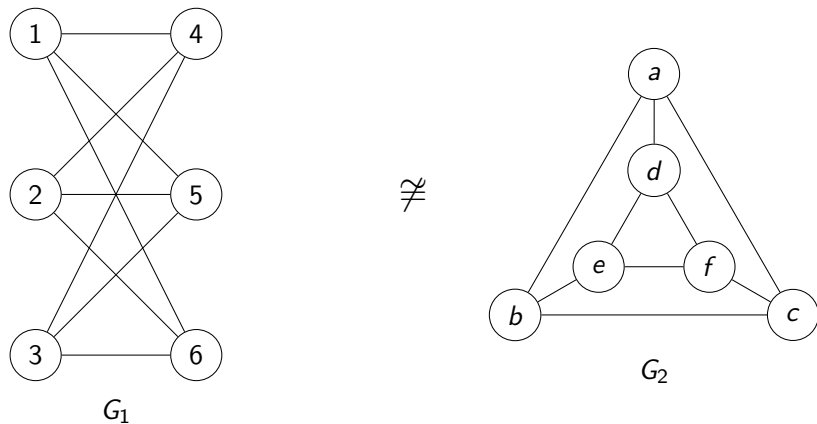
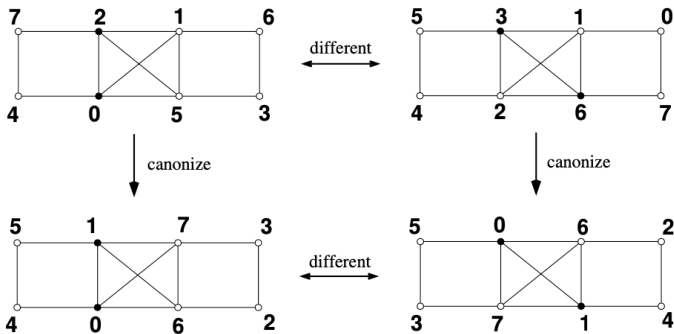


Figure 2: Pas d'isomorphisme

- Problème d'isomorphisme de graphe (IG)
- Complexité: $IG \in NP$, $IG \stackrel{?}{\in} P$, $IG \stackrel{?}{\in} NPC$
- Considéré NPI
- Meilleur algorithme quasi-polynomial: László Babai (2016)
- Forme canonique plus efficace en pratique (*Nauty*)

Forme canonique

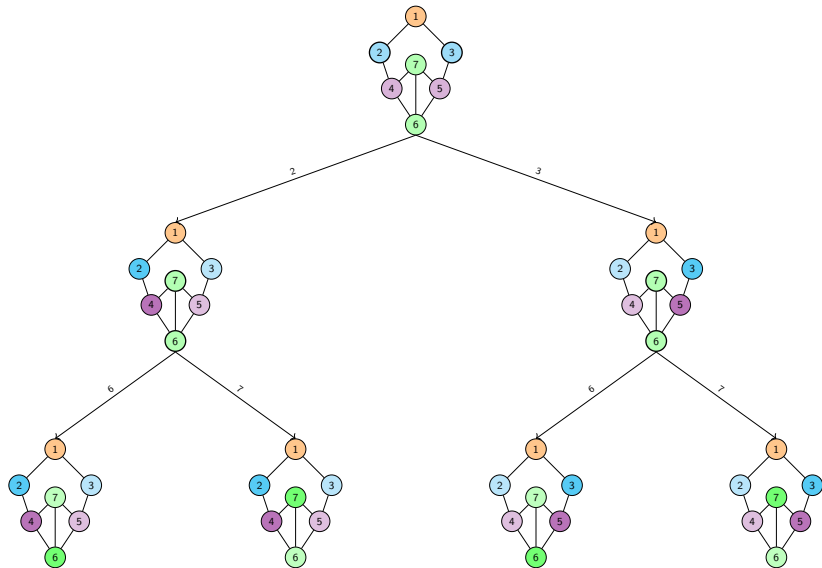
Principe: Problème d'équivalence en problème d'égalité



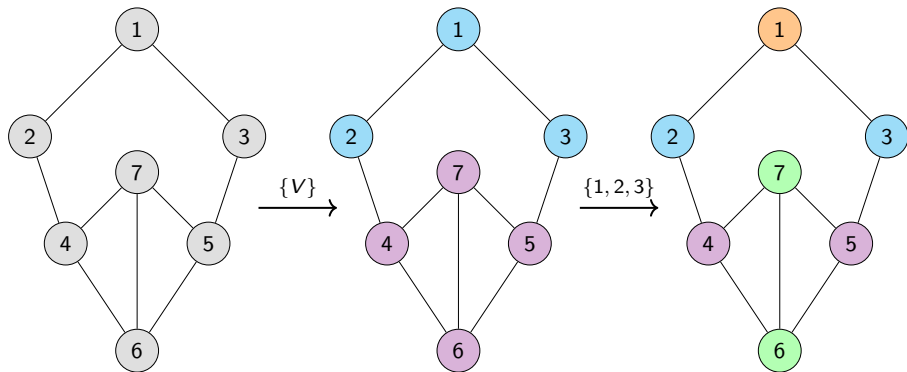
Nauty



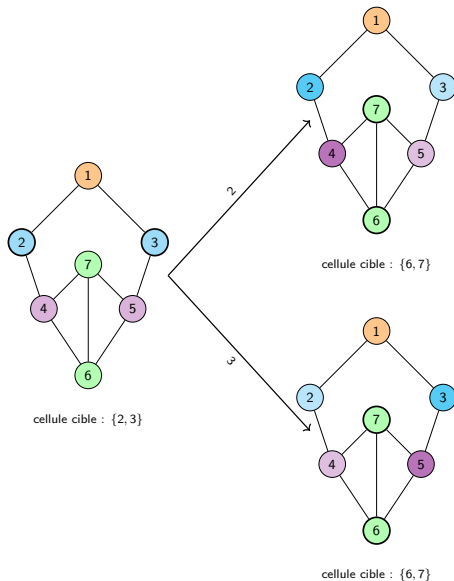
Arbre de recherche



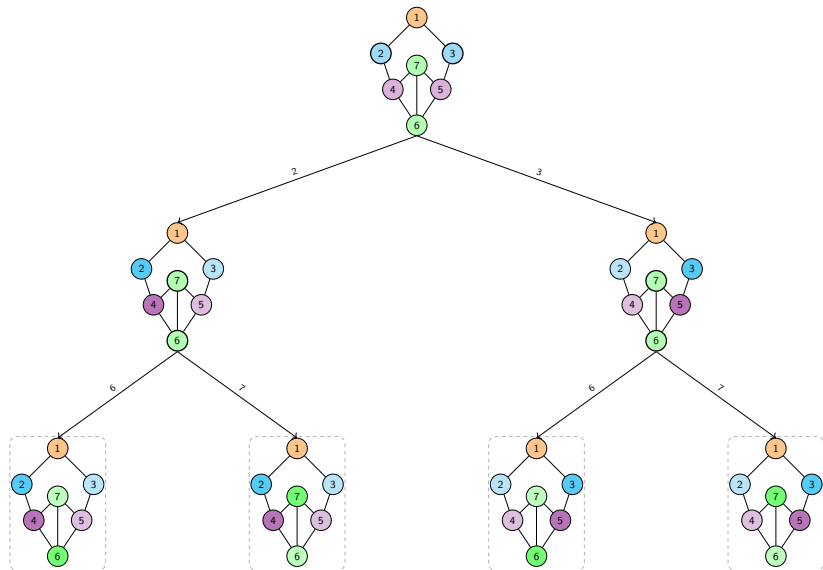
Fonction: raffinement



Fonction: sélection classe



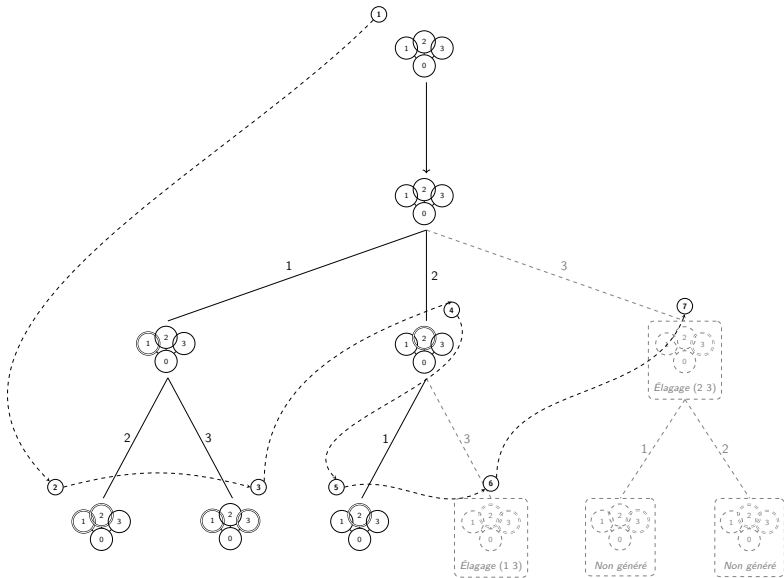
Fonction: invariant + forme canonique



Stratégies d'élagages

- 2 stratégies basées sur l'invariant
- 1 stratégie basée sur les automorphismes
- Élagage par automorphisme plus performant
- Groupe d'automorphisme en cadeau 🎁

Élagage par automorphisme



Conclusion



- *Nauty*: incontournable pour résoudre IG
- Fonctionnalités rajoutées entre temps
- Implémentation moins bonne mais respecte bien la logique

Merci pour votre attention!

Questions?

rodolphe.prevot@ulb.be

Stratégie basée sur l'invariant

$\mathcal{P}_A$

Éliminer les branches dont l'invariant ne peut pas atteindre la valeur maximale: à chaque niveau de l'arbre, on conserve le meilleur invariant trouvé jusqu'à présent, et tout autre nœud situé au même niveau dont l'invariant est strictement inférieur voit sa branche élaguée.

$\mathcal{P}_B$

Les noeuds sont comparés à un nœud de référence (noeud sur le chemin de la première feuille rencontrée). Si les deux invariants diffèrent, le sous-arbre courant est élagué. L'objectif n'est pas réellement de trouver le meilleur invariant, mais plutôt de trouver des nouveaux automorphismes.

En pratique

Il faut que les 2 conditions soient combinées pour élaguer le noeud.

Stratégie basée sur l'automorphisme

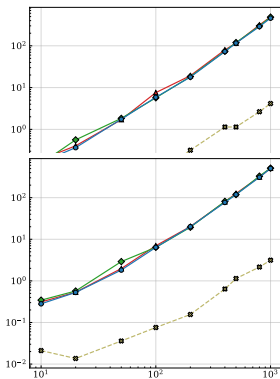
$\mathcal{P}_C$

Exploiter les automorphismes déjà découverts: si un automorphisme g associe le nœud courant σ à un nœud σ^g déjà exploré, alors les deux sous-arbres sont isomorphes. L'exploration de σ^g étant redondante, son sous-arbre est élagué.

Utilité supplémentaire

Cette stratégie permet d'avoir le groupe d'automorphisme. L'algorithme donne 2 choses: la forme canonique + groupe d'automorphisme au passage.

Seulement raffinement



Avec individualisation

